Microcontroller and Pulse-Width Modulation

An Introduction to Microcontroller and How they work

Session 01



Contents

01	**Why Electronics?**	How electronics and Control Systems are connected
02	**Microcontroller**	Microcontroller Architecture and Peripheral
03	**PWM and LEDC**	Basic of PWM and LED Control Peripheral
04	**Implementation**	Connecting the Theory to Physical System

>_

What would happen when we try to control an LED without actually understanding how electronics works?

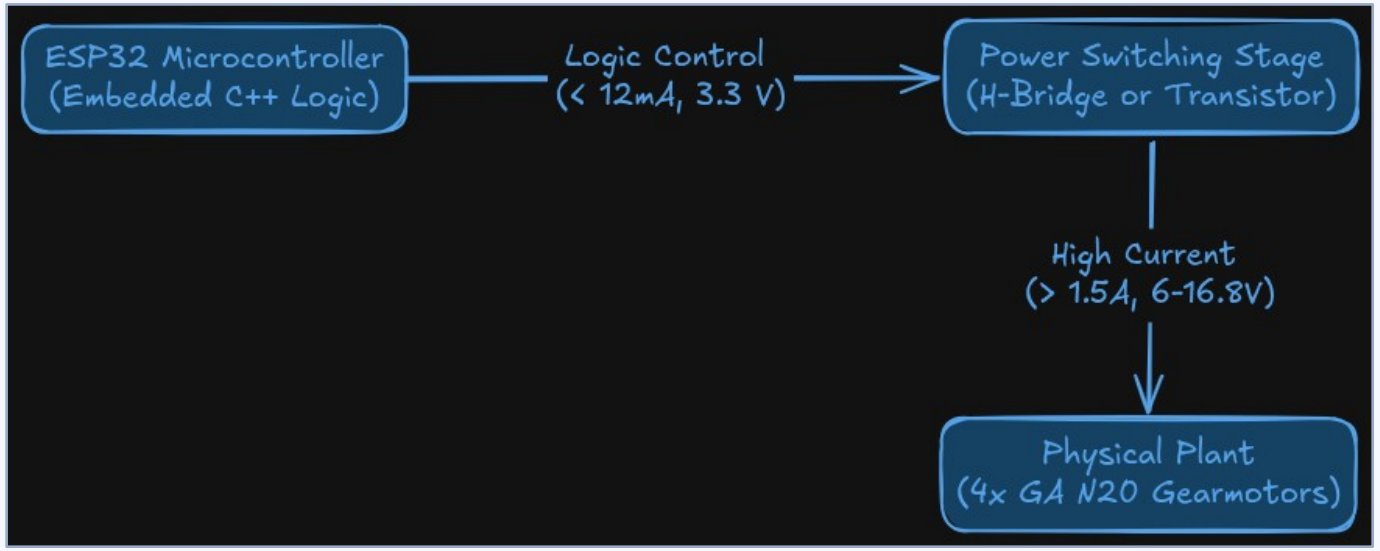
Can motors behave like an LED? Not Exactly, but Both would cause Danger!!!

The Current and Voltage Gap

What Could Go wrong?

> Electrical Destruction

- Thermal Overcurrent
- Inductive Kickback



Do NOT connect the load directly to a microcontroller pins

Logic Level Domains and Signal Translation

3.3V	**CMOS Logic Domain**	ESP32 GPIO Pins Input
5V	**TTL Domain**	Sensors and Legacy Peripheral
6V	**Motor Drive**	Isolated and Regulated Motor Driver Rail
14.8+V	**Battery Pack**	Energy Storage for all Components

Grounding, Return Paths, and Decoupling

Grounding

When 2 subsystems exchange logical signals, they must share a ground reference.

Without it the signal defined as 3.3 V by the transmitter has an undefined, floating voltage potential at the receiver.

Constraints

- **Ground loop Interference**
  - High-frequency ground bounce and inductive noise are introduced by motor
- **Separation Rule**
  - Motor path requires short paths, heavy ground wire (AWG 18 or 20), and terminated directly at power supply or screw terminal star ground
- **Decoupling Capacitors**
  - Needs capacitors (ceramic and electrolytic) to smooth high-frequency noise and buffer transient supply dips

CPU, MPU, and MCU

01 Central Processing Unit (CPU)

The ALU and instruction execution pipeline

02 Microprocessor (MPU)

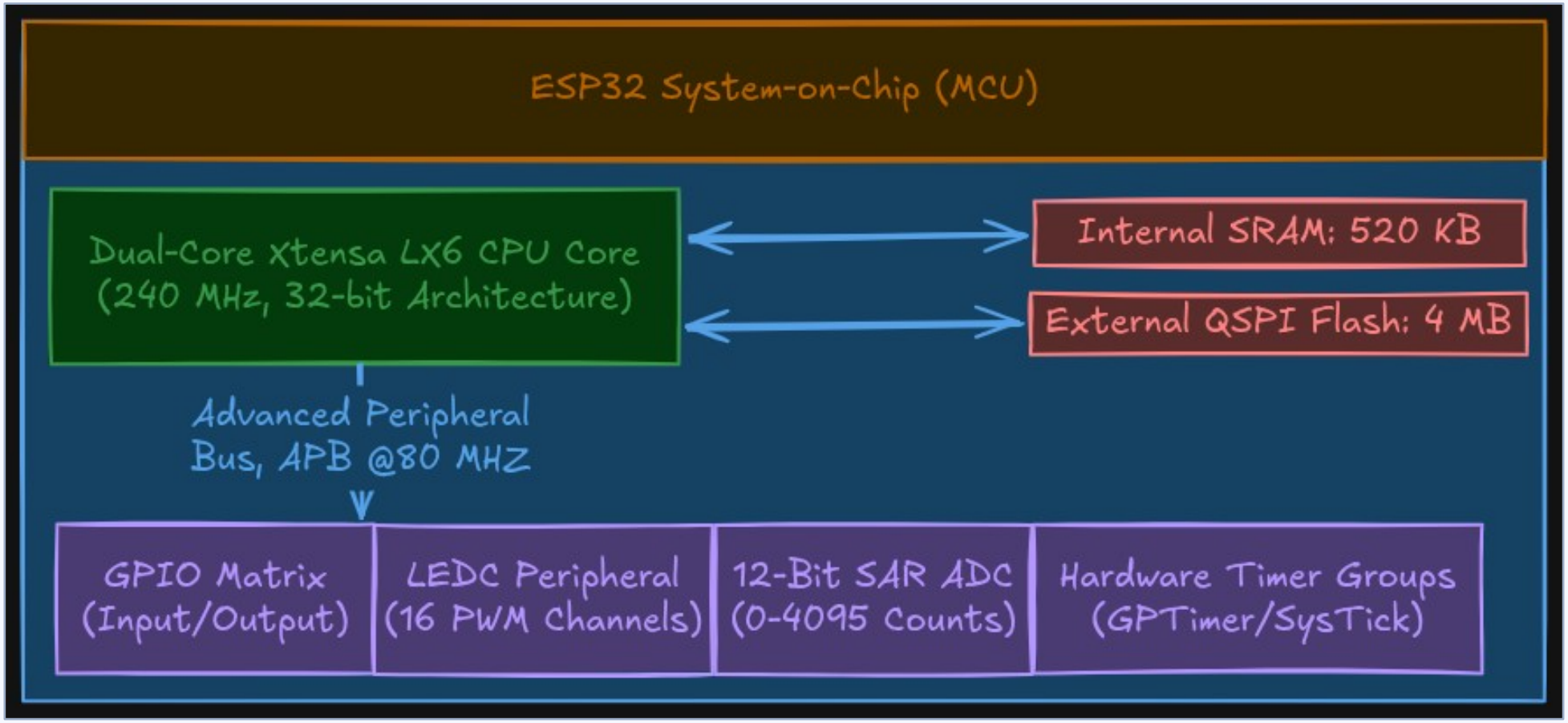
A high-performance processor, running heavy computation but lacks deterministic, hard real-time pin timing

03 Microntroller (MCU)

A self-contained SoC integrating CPU with high-speed generation and dedicated hardware peripheral, providing deterministic real-time control



ESP32 Architecture



GPIO Matrix

01 GPIO 0-33

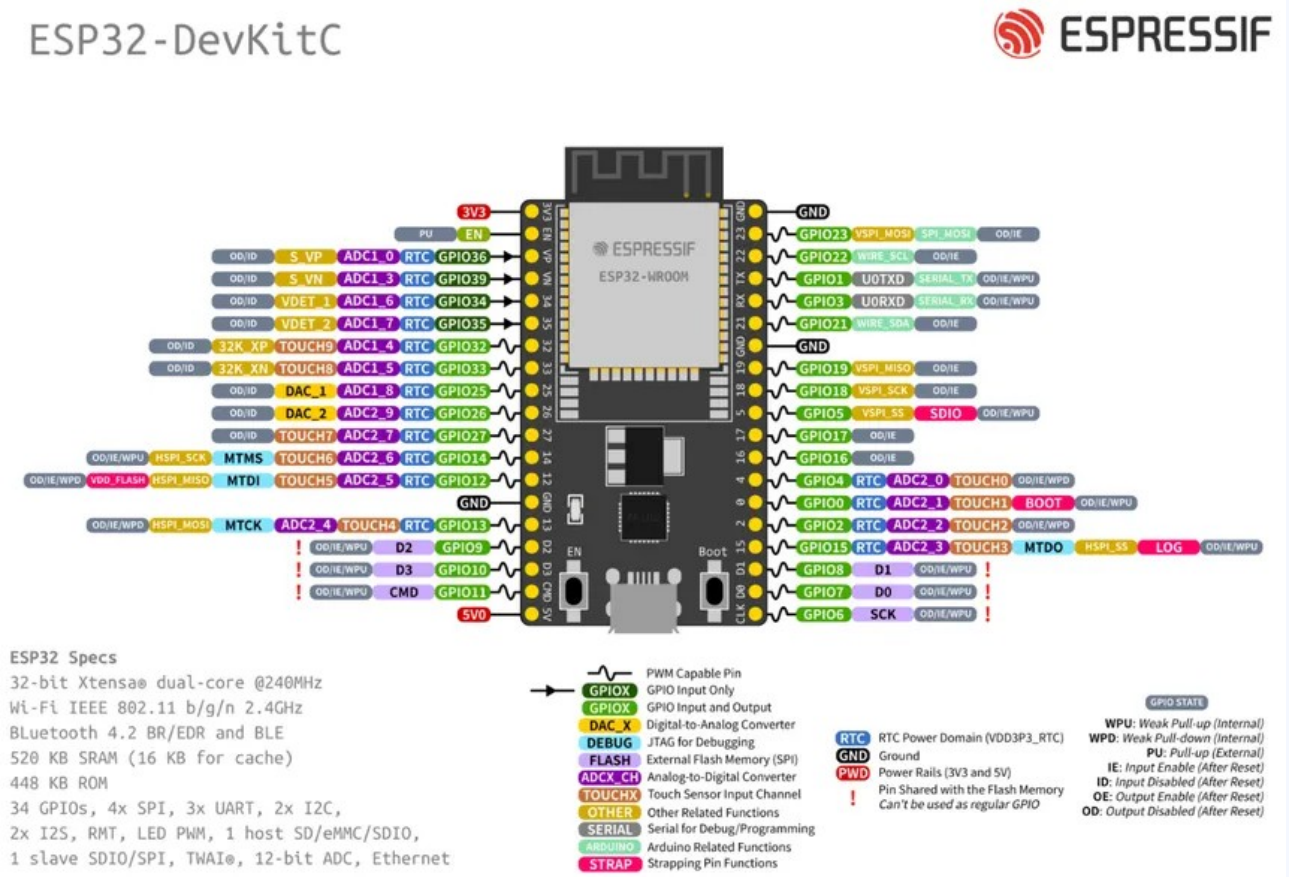
Standard bidirectional pins with fully configurable features

02 Input-Only

Lacks internal output driver and pullup or pulldown resistors

03 Boot Strapping

Determine boot mode



Floating Inputs

Floating Inputs

An input pins without pull resistors can have high-input impedance state which causes electromagnetic radiation couples capacitively into the pin gate, so the logic level oscillate unpredictably



Pull Resistors

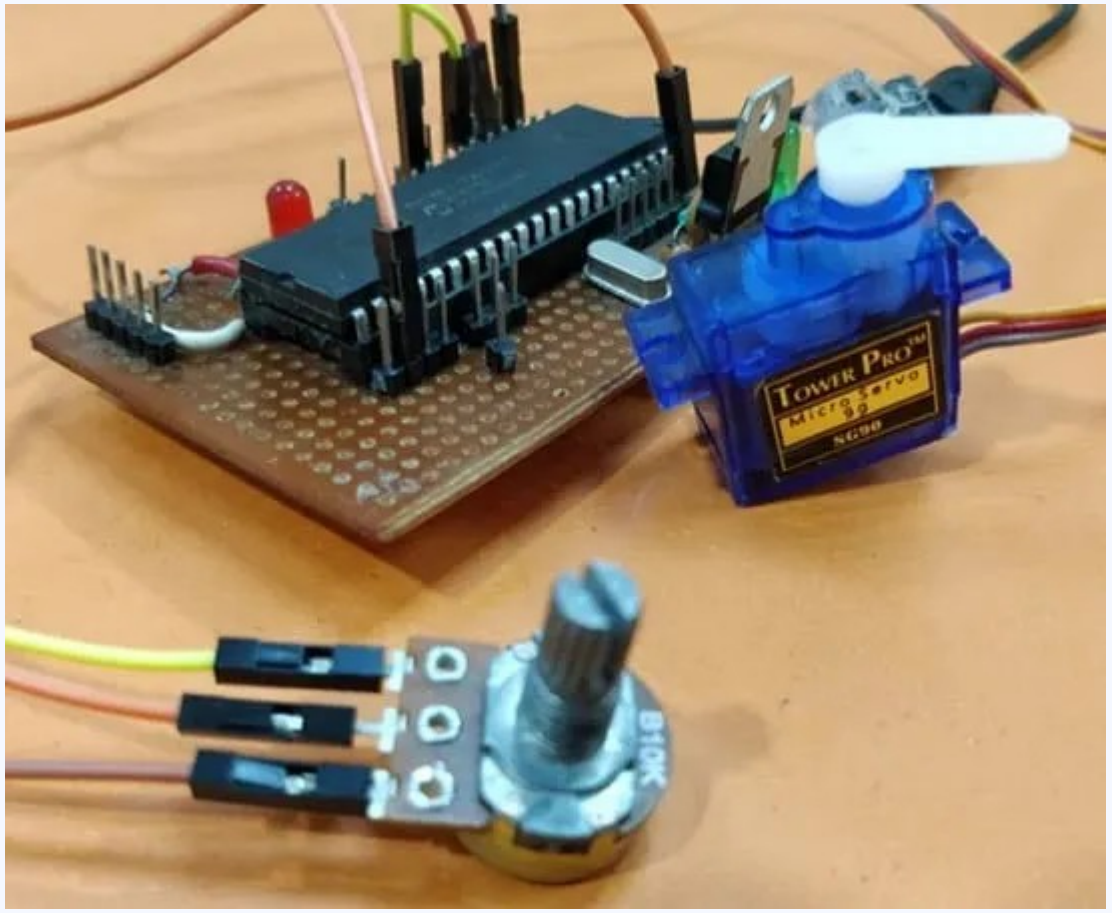
- **Input pullup**
  - Connects internal resistors to HIGH
  - pin defaults to logic HIGH
  - Ground pulls the pin to logic Low when active (active low)
- **Input pulldown**
  - Connects internal resistor to LOW
  - Pin defaults to LOW
  - When active, the pin is HIGH (active high)

What is Pulse-Width Modulation?

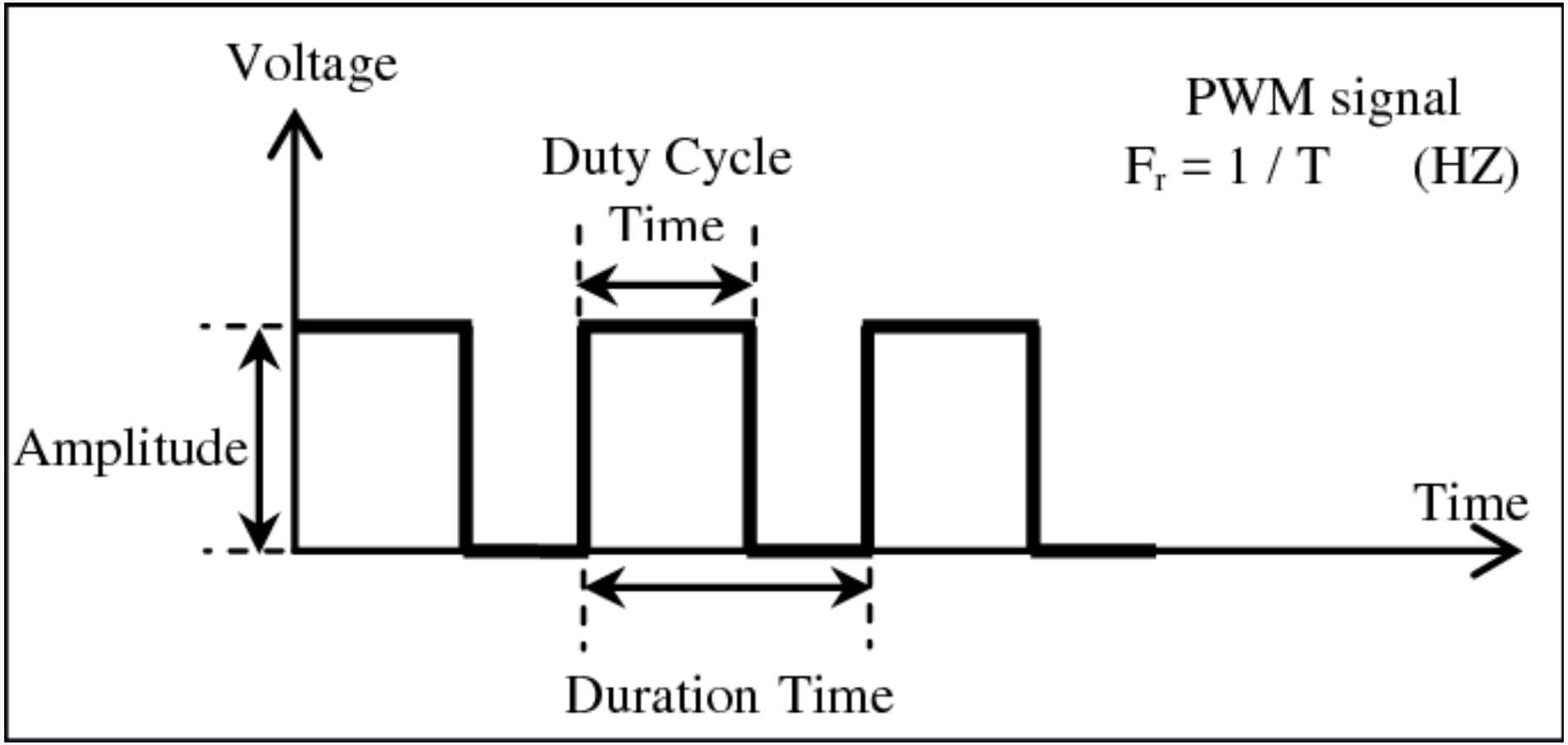
A Method for controlling analog power delivery to motors and actuators using binary digital outputs

How?

By rapidly toggle a pin between LOW and HIGH at a constant carrier frequency



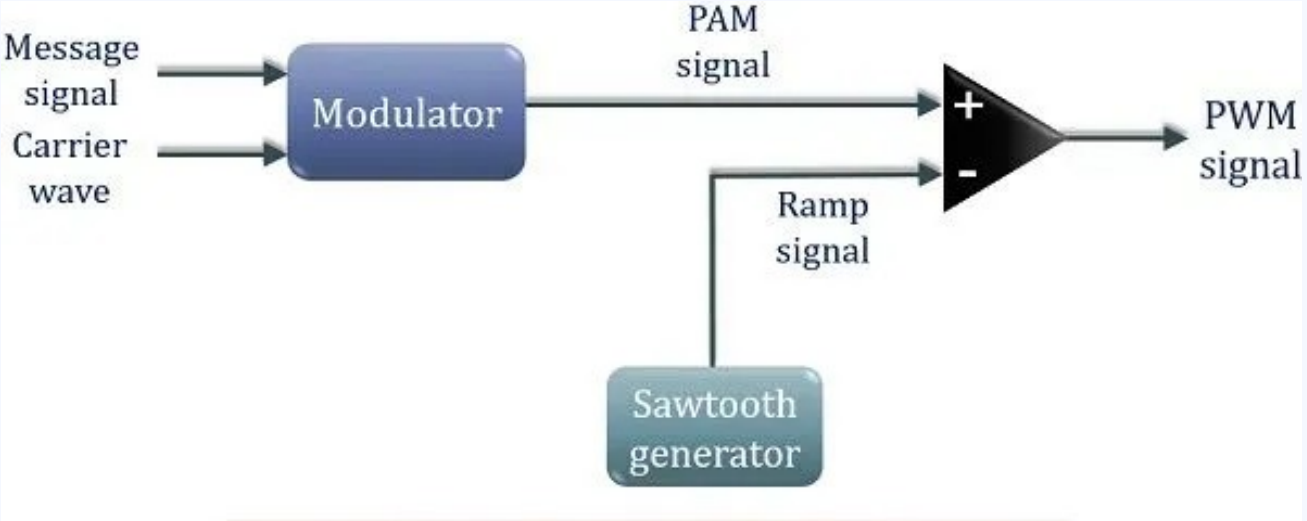
PWM Waveform



PWM Carrier Period and Carrier Frequency

$$T = T_{ON} + T_{OFF} = \frac{1}{f_{PWM}}$$

- Carrier Period: the duration of one complete waveform cycle
- Carrier Frequency: the pulse repetition rate per second



Duty Cycle and Time-Averaged DC Voltage

$$D = \frac{T_{ON}}{T}, D_{\%} = D \times 100\%$$

- Duty Cycle: the fractional ratio of active HIGH duration relative to total period
- Time-Averaged DC Voltage: the mathematical definite integral over one complete cycle

$$V_{avg} = \frac{1}{T}\int_0^T V(t)\,dt$$

$$V_{avg} = \frac{1}{T}\left(V_{HIGH} \times T_{ON} + 0 \times T_{OFF}\right)$$

$$V_{avg} = D \times V_{HIGH}$$

LEDC Hardware Peripheral

Hardware Architecture

16 independent PWM channels grouped into 2 high-speed or low-speed blocks, driven by 4 internal hardware timer modules

Clock Source

Driven by APB clock; the peripheral operates autonomously via direct memory-mapped registers, consuming zero CPU cycles

Resolution

Sets the integer quantization range of the duty cycle; maximum achievable PWM frequency is constrained by the timer clock and bit resolution

Internal Analog Filter

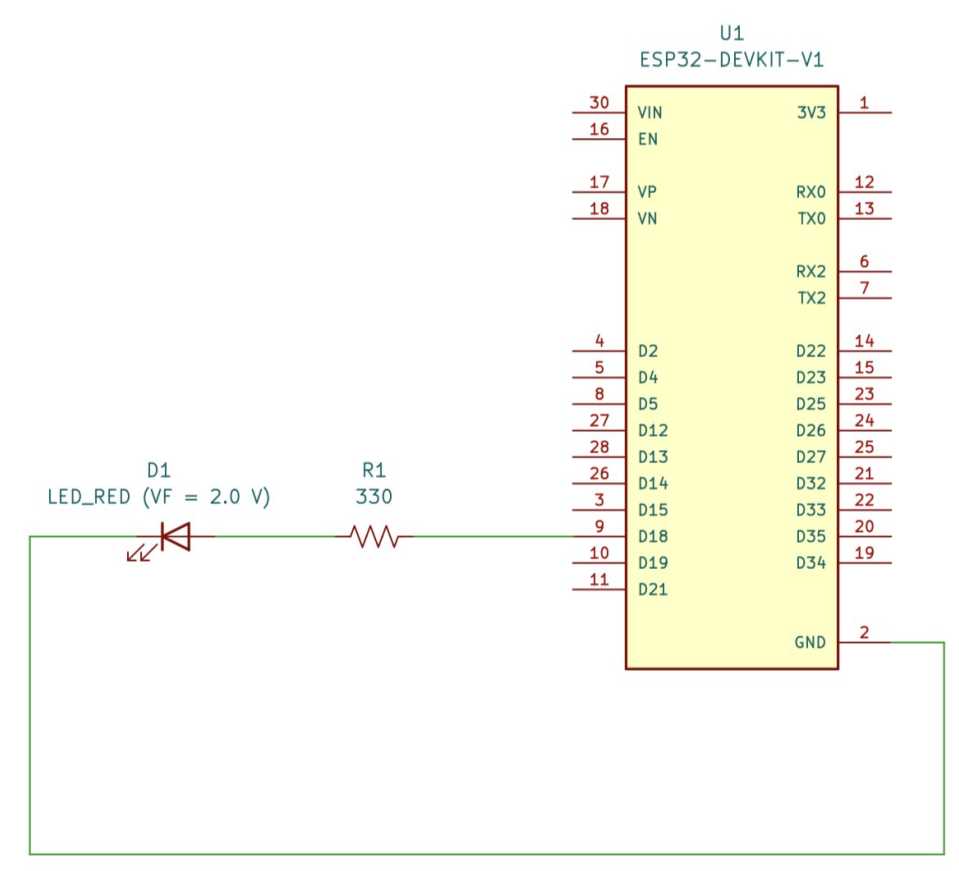
When connected to a multimeter, the internal ADC acts as a continuous low-pass filter with integration windows between 100-300 ms

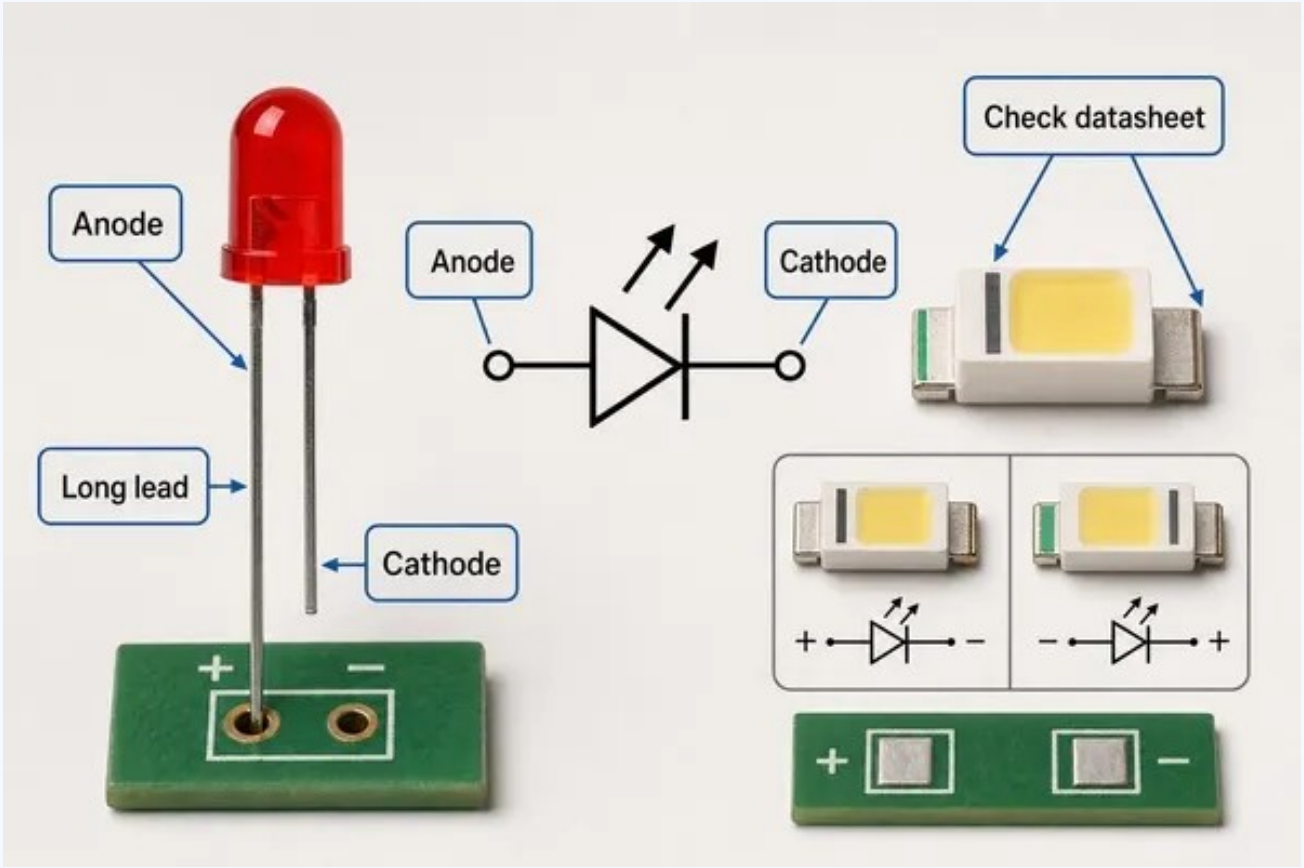
$$Steps = 2^N$$

$$V_{multimeter} = V_{avg}$$

$$V_{multimeter} = \frac{DutyCount}{255} \times 3.3\,V$$

How to wire components





How to use Multimeter

Continuity Mode	Resistance Mode	DC Voltage Mode
Keep the power disconnected	Keep the power disconnected	Connect to power
Verify 0 Ohm continuity along GND	Verify passive component values	Verify LDO regulator and buck converter rail
Confirm zero short-circuit between VCC and GND	Verify DC motor armature coil winding resistance	Probe PWM output pin to verify voltage scaling

Pre-Power Test

01 Continuity Mode

Confirm that audible beep is working by touching the red and black probes together

02 Power Check

Check metal USB shield and the GND rail; confirm they are shorted

03 Resistance Mode

Measure between the 3V3 pin and GND; if the reading is below 10 Ohms, do NOT connect to power

Do NOT plugin the USB cable until you completed these tests

Firmware: Configure PWM

```
const uint8_t PWM_OUTPUT_PIN = 18;        // GPIO connected to the
LED or driver input
const uint32_t PWM_FREQUENCY = 20000;     // 20 kHz ultrasonic PWM
carrier
const uint8_t PWM_RESOLUTION = 8;         // 8-bit duty resolution:
0-255 counts
const uint8_t LEDC_CHANNEL = 0;            // Hardware LEDC channel
index
```

GPIO 18 | 20 kHz carrier | 8-bit duty | LEDC channel 0

Firmware: Initialize and Write

```
bool initHardwarePWM() {
  return ledcAttachChannel(
    PWM_OUTPUT_PIN,
    PWM_FREQUENCY,
    PWM_RESOLUTION,
    LEDC_CHANNEL
  );
}

void setHardwarePWM(uint8_t duty_count) {
  ledcWrite(PWM_OUTPUT_PIN, duty_count);
}
```

Initialize once in setup(); write a new duty value through setHardwarePWM().

Firmware: Update Every 20 ms

```
static uint32_t previous_ramp_time_ms = 0;

const uint32_t current_time_ms = millis();

// Unsigned subtraction remains correct when millis() rolls over
if ((uint32_t)(current_time_ms - previous_ramp_time_ms) >=
    RAMP_INTERVAL_MS) {

  previous_ramp_time_ms += RAMP_INTERVAL_MS;
```

Inside loop(): check elapsed time, then enter the ramp update block.

Firmware: Apply, Reverse, and Step

```
setHardwarePWM(current_duty_count);

// Reverse direction at the endpoints of the ramp
if (current_duty_count == 255) {
  ramp_direction = -1; // Begin deceleration
} else if (current_duty_count == 0) {
  ramp_direction = 1;  // Begin acceleration
}

// Update the duty count without allowing unsigned underflow
current_duty_count = static_cast<uint8_t>(
  static_cast<int16_t>(current_duty_count) + ramp_direction
);
```

Inside the timed block: apply duty, reverse at the endpoints, then update.

Verify Data

01 Upload firmware

Connect the ESP32 to the PC; upload the code; open Serial Monitor at 115200 baud

02 Measure the Voltage with Multimeter

Use DC Voltage Mode (20 V range); check power rail, GPIO 18, and resistor

03 Observe and Compare

Compare the Serial Monitor Output with Multimeter readings

Measured Voltages MUST be around the same as theoretical prediction with minor deviations

Debugging

LED Remains Dark

Measure voltage across the LED leads; if it reads 1.8-2V but emits no light, the diode is reversed

If the voltage across the LED is 0V, check MCU pin; if it's toggling, the circuit is open

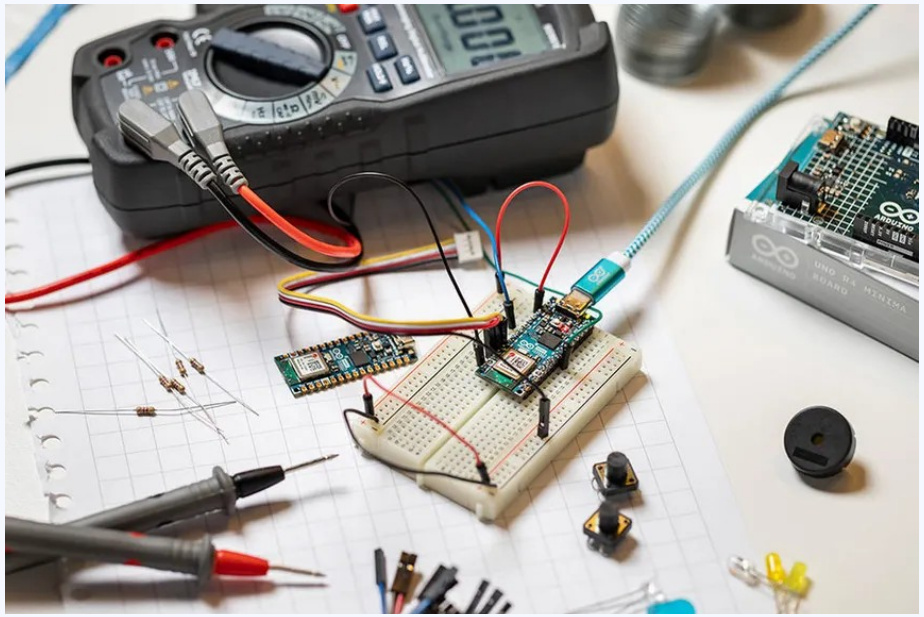
Serial Monitor Output Unreadable Text or Blank

Verify baud rate

Press the physical EN button the ESP32 board to restart the processor

IDE Reports: Failed to connect to ESP32

Hold down the physical BOOT button on the ESP32 during "Connecting….." compilation phase until writing begins, then release



References

Making Embedded Systems: Design Patterns for Great Software

Elecia White

The Art of Electronics: Student Manual

Thomas C. Hayes and Paul Horowitz

Feedback Systems: An Introduction for Scientists and Engineers

Karl Johan Åström and Richard M. Murray

Modern Robotics: Mechanics, Planning, and Control

Kevin M. Lynch and Frank C. Park

ESP32 Arduino Core Documentation: LED Control (LEDC)

Espressif Systems

ESP32 Arduino Core Documentation: GPIO

Espressif Systems

Arduino Core for the ESP32

Espressif Systems

Motor Drivers Architecture and Overview

Texas Instruments

Thank you!

Questions & discussion



CONNECT / CONFIGURE / MEASURE